

Statistical Simulation in ngspice

Monte Carlo, Latin-Hypercube, high-sigma, correlations, and yield

Ngspice + OpenVAF Enhancements project

July 2026

Contents

Statistical simulation in ngspice — a complete guide	1
TL;DR — what's available	1
1. Random parameters	2
2. The two Monte Carlo idioms	2
3. Latin-Hypercube sampling — <code>mcsample</code>	3
4. High-sigma rare events — <code>highsigma</code>	4
5. Process/mismatch correlations — <code>mccorr</code> + <code>mvnorm</code>	5
6. Yield — <code>montecarlo</code>	5
7. Corners, and combining everything	6
8. Practical guidance	6
Reproducing the figures	7

Statistical simulation in ngspice — a complete guide

This build of ngspice-46 has a full statistical-analysis suite: ordinary Monte Carlo, Latin-Hypercube low-discrepancy sampling, high-sigma rare-event estimation, native process/mismatch correlations, and a packaged yield command. This note documents all of it end-to-end — the distribution functions, the sampling controls, the commands, and worked, plotted examples — so you can go from "I have a Verilog-A/SPICE circuit" to "here is its yield, and its 5-sigma failure probability, with correlated device statistics."

Everything here is a front-end capability: it changes only which values the random `.params` take, not the circuit solve, so it works identically under the Sparse 1.3 and KLU linear solvers and with built-in and OSDI/Verilog-A devices.

TL;DR — what's available

Capability	Command / syntax	Enhancement
Random parameters	<code>agauss / gauss / aunif / unif / limit in .param</code>	base
Deterministic seeding	<code>setseed <s></code>	base
Ordinary Monte Carlo	the reset loop, or the alter loop	base (E-66)
Latin-Hypercube sampling	<code>mcsample lhs <N> [seed <s>]</code>	E-149
High-sigma rare events	<code>highsigma <N> -metric <e> -max/-min <spec></code>	E-150
Process/mismatch correlations	<code>mccorr <k> <matrix> + mvnorm(i)</code>	E-151

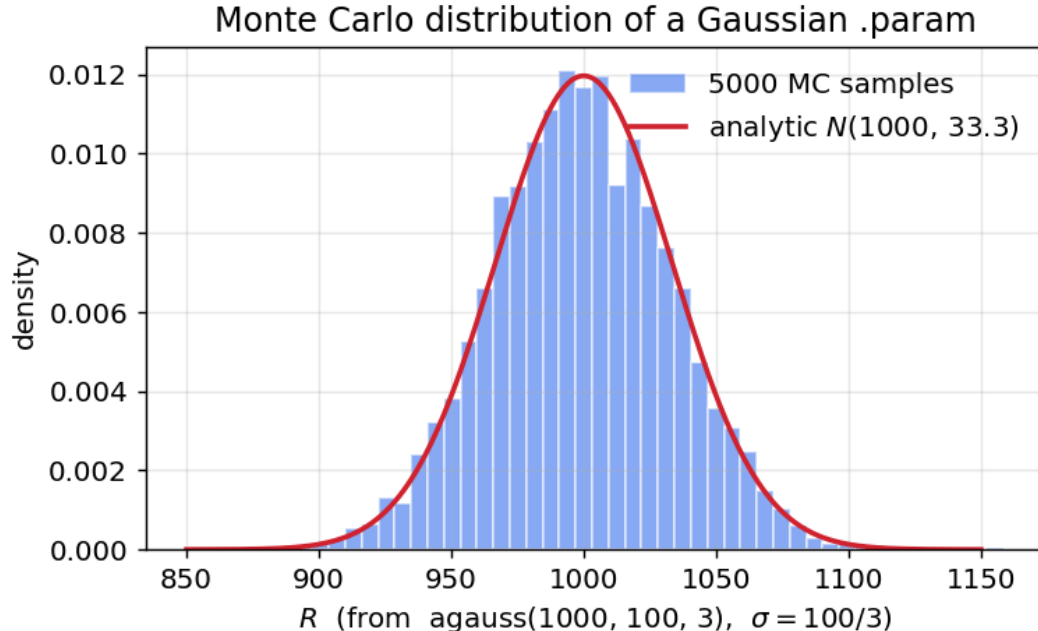
Capability	Command / syntax	Enhancement
Packaged yield	montecarlo <N> [-lhs] -spec <e> -max/-min <spec>	E-151
Process corners	.lib / .include corner model sets	base

1. Random parameters

Statistical variation enters through the random functions in a `.param` expression. They are evaluated once per Monte Carlo sample (once per reset re-source), so each sample re-throws the dice:

function	draw	notes
agauss(nom, avar, sig)	Gaussian, mean nom, $\sigma = \text{avar}/\text{sig}$	"absolute": avar is the sig- σ spread
gauss(nom, rvar, sig)	Gaussian, $\sigma = \text{nom} \cdot \text{rvar}/\text{sig}$	"relative" variation
aunif(nom, avar)	uniform on [nom-avar, nom+avar]	
unif(nom, rvar)	uniform on [nom(1-rvar), nom(1+rvar)]	
limit(nom, avar)	nom $\pm$ avar (a fair coin)	corner-style $\pm$

So `agauss(1000, 100, 3)` is a resistor with a nominal 1 k Ω and a 3- σ spread of 100 Ω , i.e. $\sigma = 33.3 \Omega$. Drawing 5000 of them and histogramming recovers the Gaussian exactly:



The independence gotcha. Every *textual* occurrence of a random {param} draws independently — two devices written with the same {rr} get *different* values in one sample. Matched/correlated devices need either the shared-`.param` idiom or the native correlation support in §5.

2. The two Monte Carlo idioms

(a) The **reset** idiom — a random `.param` feeds a device/model, and each reset re-resources the deck (re-throwing the dice) and re-runs the analysis:

```

* reset-idiom Monte Carlo
.param rr = agauss(1000, 100, 3)
V1 a 0 DC 1
R1 a 0 {rr}
.control
  setseed 1
  let n = 500
  let iv = unitvec(n)
  let run = 0
  dowhile run < n
    reset          ; re-throws rr, re-runs
    op
    let iv[run] = i(V1)
    let run = run + 1
  end
  print mean(iv) stddev(iv)
.endc
.end

```

This is the idiom every command below builds on. It works with `.model-card` parameters (`r={rr}`) and OSDI/Verilog-A instances alike.

(b) The **alter** loop — control-language random vectors (`sgauss(0)`, `sunif(0)`) assigned per run with `alter`; no netlist re-parse, so it is faster, but it does not benefit from `mcsample`/`highsigma`/`mccorr`, which target the `.param` idiom (that is where the well-defined per-sample boundary is).

`setseed <s>` makes either idiom bit-for-bit reproducible.

3. Latin-Hypercube sampling — **mcsample**

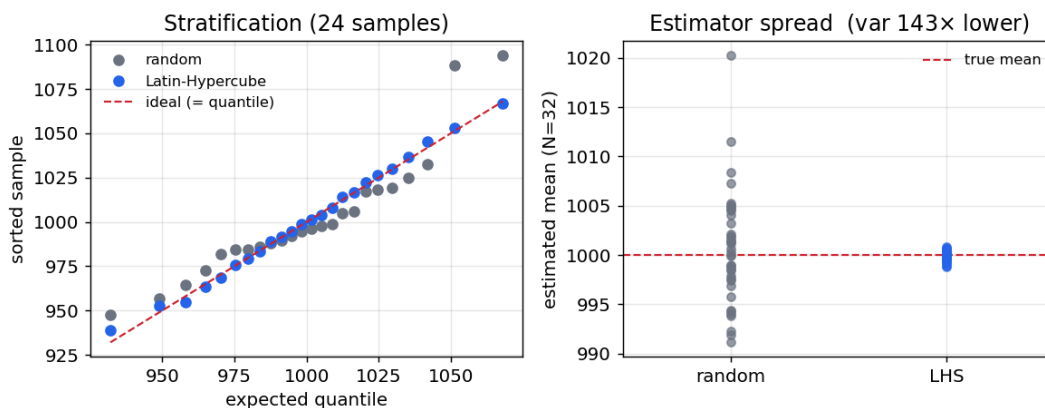
Plain Monte Carlo draws each parameter independently, so a modest run count clumps in some regions and leaves gaps in others, and the estimate converges only as $1/\sqrt{N}$. `mcsample lhs <N>` switches the `.param` draws to Latin-Hypercube sampling: each random dimension's range is split into N equal-probability strata and hit exactly once (with an independent stratum permutation per dimension; Gaussians are stratified in probability space through the inverse-normal CDF).

```

mcsample lhs <N> [seed <s>]   engage LHS for the next N reset-driven samples
mcsample random | off          revert to independent draws

```

The left panel shows the stratification (the LHS samples land on their quantiles; the random ones scatter); the right shows the payoff — the spread of the estimated mean over many trials collapses (here $\sim 100\times$ lower variance at the same N):



Usage is a one-line change to the idiom above:

```
.control
  mcsample lhs 200 seed 1          ; <-- the only change
  let run = 0
  dowhile run < 200
    reset
    op
    let iv[run] = i(V1)
    let run = run + 1
  end
  print mean(iv) stddev(iv)       ; a much tighter estimate than plain MC
.endc
```

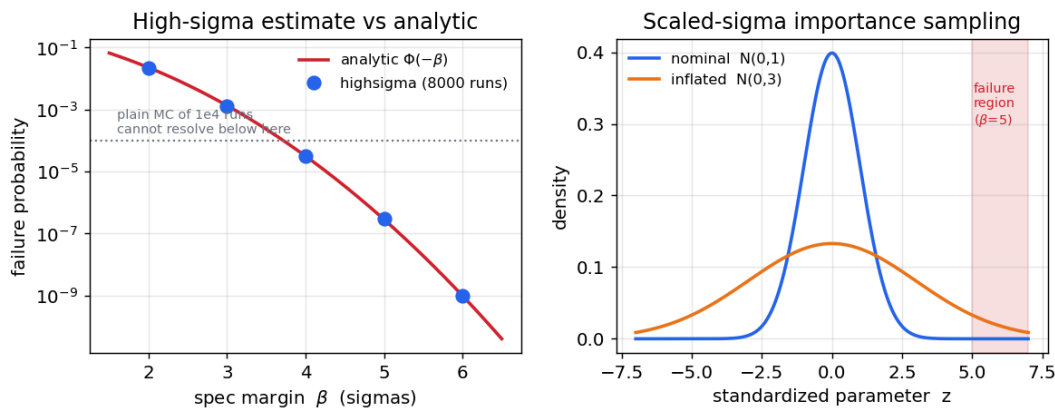
Use LHS whenever you want an accurate *whole-distribution* statistic (mean, stddev, a moderate quantile) for a given run budget.

4. High-sigma rare events — **highsigma**

For high-replication circuits (an SRAM cell instanced millions of times), the number that matters is a 4–6 sigma failure probability of $1e-7 \dots 1e-9$. Plain Monte Carlo would need $1e7 \dots 1e9$ runs just to see a handful of failures. **highsigma** estimates such probabilities with a few thousand runs by scaled-sigma importance sampling: it inflates every Gaussian `.param`'s σ by a factor λ so the failure region is sampled often, then reweights each sample by the likelihood ratio $p_{\text{nominal}}/p_{\text{inflated}}$ to keep the estimate unbiased. It is direction-free — no gradient or worst-case-distance search.

```
highsigma <N> [-scale <lambda>] [-seed <s>] [-analysis <cmd>]
             -metric <expr> [-max <hi>] [-min <lo>]
```

The left panel shows the estimate tracking the analytic $\Phi(-\beta)$ from 2 to 6 sigma, far below what plain MC of the same budget can resolve; the right panel shows the mechanism — the inflated distribution reaches the tail the nominal one never does:



* probability that R exceeds a 4.5-sigma spec (1150 ohm); analytic $3.4e-6$

```
.param rr = agauss(1000, 100, 3)
V1 a 0 DC 1
R1 a 0 {rr}
.control
  highsigma 6000 -scale 3.0 -seed 1 -analysis op -metric -1/i(v1) -max 1150
.endc
.end
```

```
P(fail)          : 3.2349e-06 +/- 3.26e-07 (relative error 10.1%)
equivalent sigma : 4.510 (one-sided, P = Phi(-sigma))
```

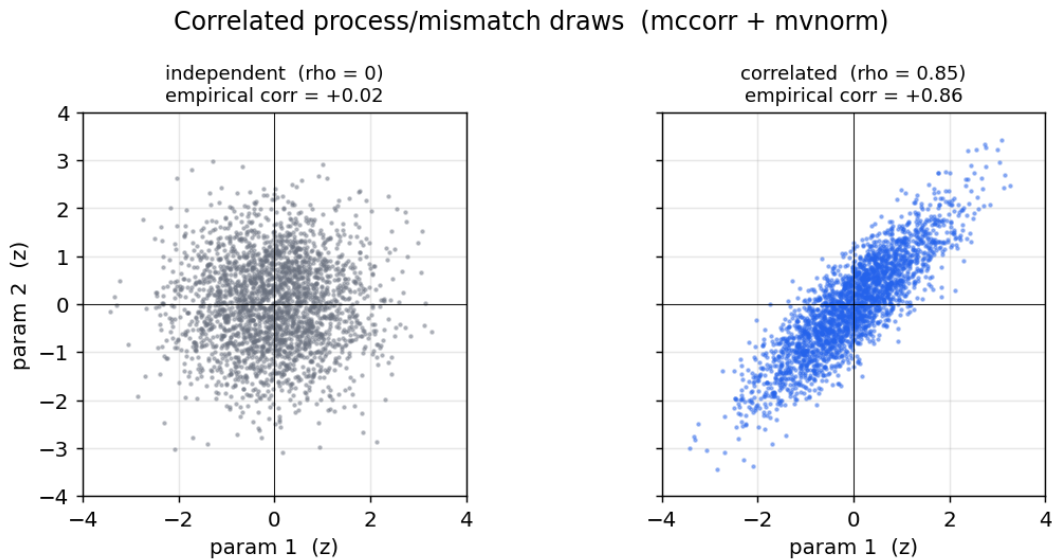
The spec is `-max/-min` numeric limits rather than a `>/<` inside the metric, because a bare `>` in a control command is an I/O redirect. Results are also left in `highsigma_pfail`, `highsigma_relerr`, `highsigma_sigma`, `highsigma_nfail`.

5. Process/mismatch correlations — **mccorr** + **mvnorm**

Whether two devices' variation is *correlated* is often what decides the yield — but plain MC draws every `agauss` independently. `mccorr` registers a $k \times k$ correlation matrix (Cholesky-factored once, and rejected if not positive-definite), and the new `.param` function **mvnorm(i)** returns the *i*-th component of one correlated standard-normal draw per sample ($y = L \cdot z$):

```
mccorr 2 1 0.85 0.85 1 ; rho = 0.85 between the two factors
.param r1 = 1000 + 100*mvnorm(1) ; r1, r2 now vary together
.param r2 = 1000 + 100*mvnorm(2)
```

`mvnorm(i)` returns a unit-variance standard normal (scale it in the `.param`); the matrix is a correlation matrix (unit diagonal). Drawing two parameters at $\rho=0$ vs $\rho=0.85$ shows the joint distribution tilt from a round blob to an elongated one:



The underlying *z*'s are drawn through the same sampler `mcsample/highsigma` use, so correlations compose with Latin-Hypercube and importance sampling automatically. The classic process + mismatch decomposition is just:

```
mccorr 1 1 ; one shared process factor
.param vth1 = 0.5 + 0.03*mvnorm(1) + 0.01*agauss(0,1,1) ; global + local
.param vth2 = 0.5 + 0.03*mvnorm(1) + 0.01*agauss(0,1,1) ; shares the process term
```

Here `mvnorm(1)` (the global process shift) is shared by both devices, while each `agauss(0,1,1)` is independent local mismatch — exactly the standard model.

6. Yield — **montecarlo**

`montecarlo` packages the whole flow: run the samples, apply the pass/fail specs, report the yield with a confidence interval.

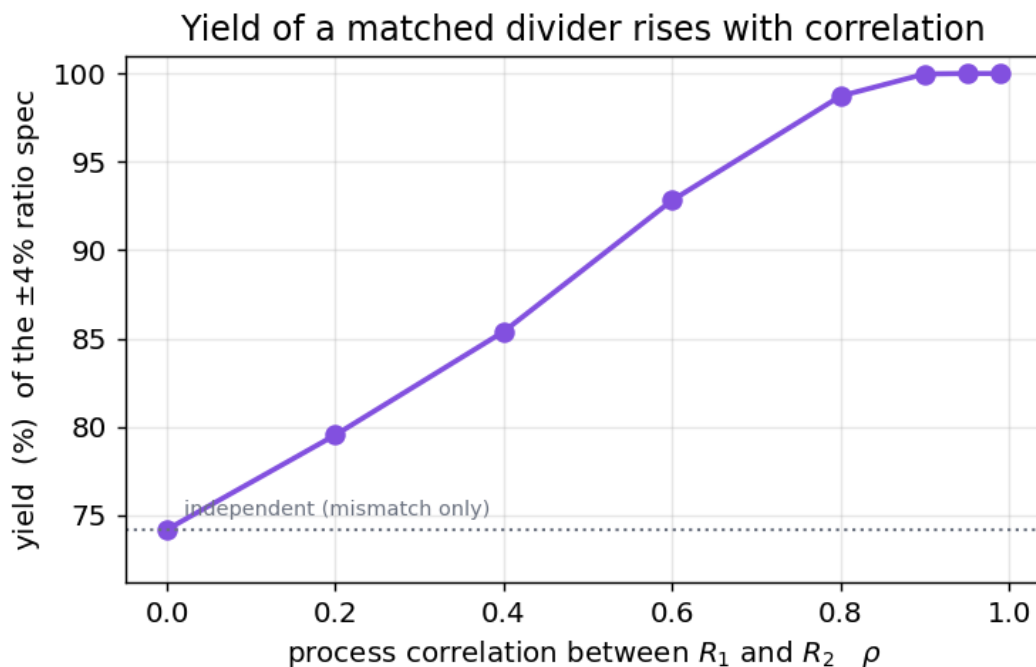
```
montecarlo <N> [-lhs] [-seed <s>] [-analysis <cmd>]
            (-spec <metric> [-max <hi>] [-min <lo>])...
```

A sample passes only if *every* spec's metric is within its `-max/-min` limits. It reports the yield with a

Wilson 95% confidence interval and a per-spec violation count, and leaves `montecarlo_yield`, `montecarlo_npass`, `montecarlo_n` for scripting. `-lhs` gives a much lower-variance yield estimate.

```
* a matched divider, +/-4% ratio spec
.param r1 = 1000 + 50*mvnorm(1)
.param r2 = 1000 + 50*mvnorm(2)
V1 in 0 DC 1
R1 in out {r1}
R2 out 0 {r2}
.control
  mccorr 2 1 0.9 0.9 1
  montecarlo 5000 -lhs -analysis op -spec v(out) -max 0.52 -min 0.48
.endc
.end
```

Because `mvnorm` correlations feed straight into `montecarlo`, the yield of a *matched* pair depends strongly on how correlated the two devices are — from ~74 % when independent to ~100 % when they track each other. Getting the correlation model wrong grossly misestimates the yield:



7. Corners, and combining everything

Process corners (TT / FF / SS / ...) are the ordinary ngspice `.lib/.include` mechanism — a corner model set is selected by a `.lib "models.lib" ff` line. Corners compose with all of the above: load a corner, then run `montecarlo` at that corner to get the corner's yield, or loop the corners around the MC. A typical production flow is therefore:

1. `.lib` in a process corner (or loop over corners);
2. declare correlated process/mismatch with `mccorr + mvnorm`;
3. `montecarlo ... -lhs` for the yield (or `highsigma ...` for a rare-event spec).

8. Practical guidance

- Reproducibility. Every run is seeded (`setseed`, or the command's `seed/ -seed`); the same seed reproduces the sample sequence bit-for-bit, so results are portable and regressions are detectable.

- Which sampler? Use plain MC to sanity-check; **mcsample lhs** (or **montecarlo -lhs**) for an accurate mean/stddev/yield at a fixed budget; **highsigma** when the interesting probability is out in the 4–6 σ tail.
- Choosing λ for highsigma: ~ 2 for 3 σ , ~ 2.5 for 4 σ , ~ 3 for 5–6 σ . Watch the reported relative error — if it is large, raise N or λ .
- Correlations matter. For matched devices, a plausible correlation is worth far more than more samples: as the yield figure shows, ρ can move the yield by tens of percent.
- Cost. Each sample re-sources the deck (a few ms for a small deck), so a yield/rare-event run is thousands of re-sources — heavy but far cheaper than the 10^7 – 10^9 plain-MC runs a 5- σ estimate would otherwise need.

Reproducing the figures

`make_statistics_figs.py` (in this folder) regenerates every plot from real ngspice runs — it drives the committed binary through the `mcsample`, `highsigma`, `mccorr`, and `montecarlo` commands and plots the results with matplotlib. See the per-feature example suites for the verifiers: [lhs_examples](#), [highsigma_examples](#), [yield_examples](#), and [montecarlo_examples](#).